Computational Statistics - Coursework

MSc in Statistics 2025/26, Imperial College London

06046958

I, 06046958, certify that this assessed coursework is my own work, unless otherwise acknowledged, and includes no plagiarism. I have not discussed my coursework with anyone else except when seeking clarification with the module lecturer via email or on MS Teams. I have not shared any code underlying my coursework with anyone else prior to submission.

Question 1

(a)

We discuss three cases for $\sigma = (1, 10, 20, 30)^T$ and choose the start points at $x_0 = (2, 20, 30)^T$, and display the results respectively:

Mixing quality:

```
Chain 1 : Sigma is 1 , ESS is: 8466.651
Chain 2 : Sigma is 1 , ESS is: 5930.458
Chain 3 : Sigma is 1 , ESS is: 17351.88
Chain 4 : Sigma is 10 , ESS is: 329.9607
Chain 5 : Sigma is 10 , ESS is: 296.8636
Chain 6 : Sigma is 10 , ESS is: 347.3938
Chain 7 : Sigma is 20 , ESS is: 1337.624
Chain 8 : Sigma is 20 , ESS is: 1533.531
Chain 9 : Sigma is 20 , ESS is: 1501.018
Chain 10 : Sigma is 30 , ESS is: 1513.605
Chain 11 : Sigma is 30 , ESS is: 1492.317
Chain 12 : Sigma is 30 , ESS is: 1573.973
```

Traceplots:

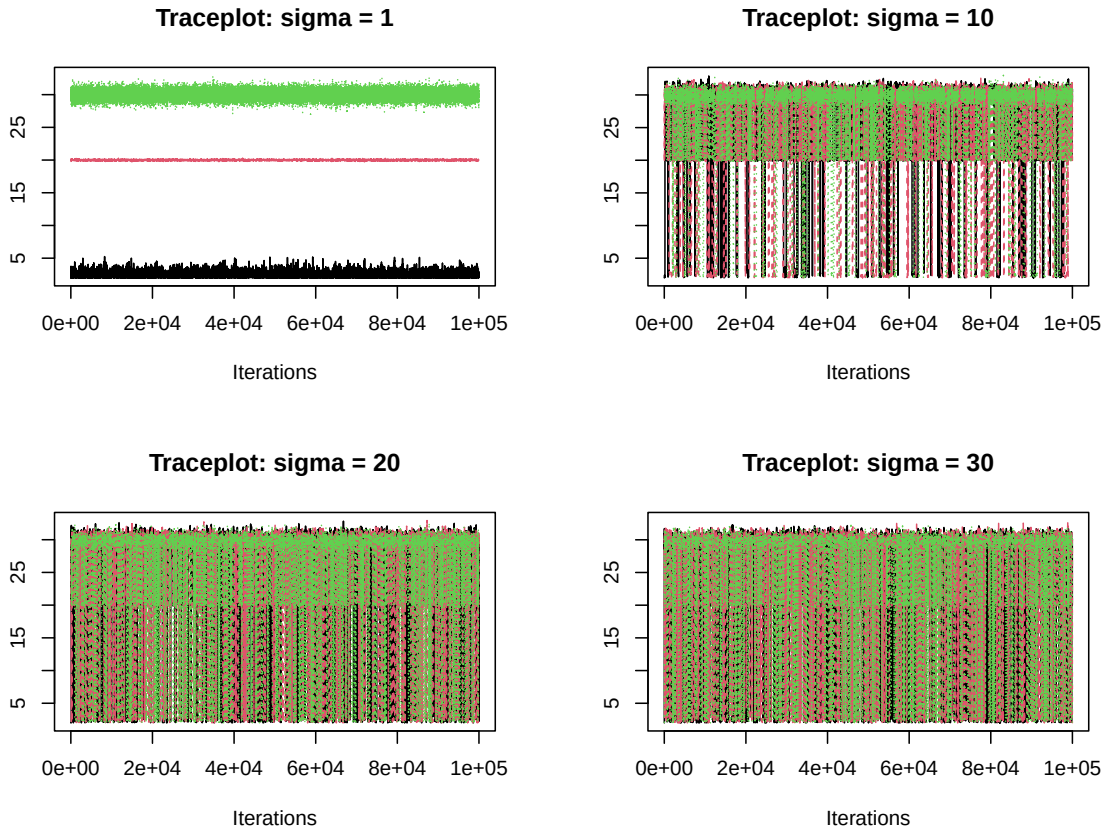


Figure 1: Traceplot for four different variances

Convergence:

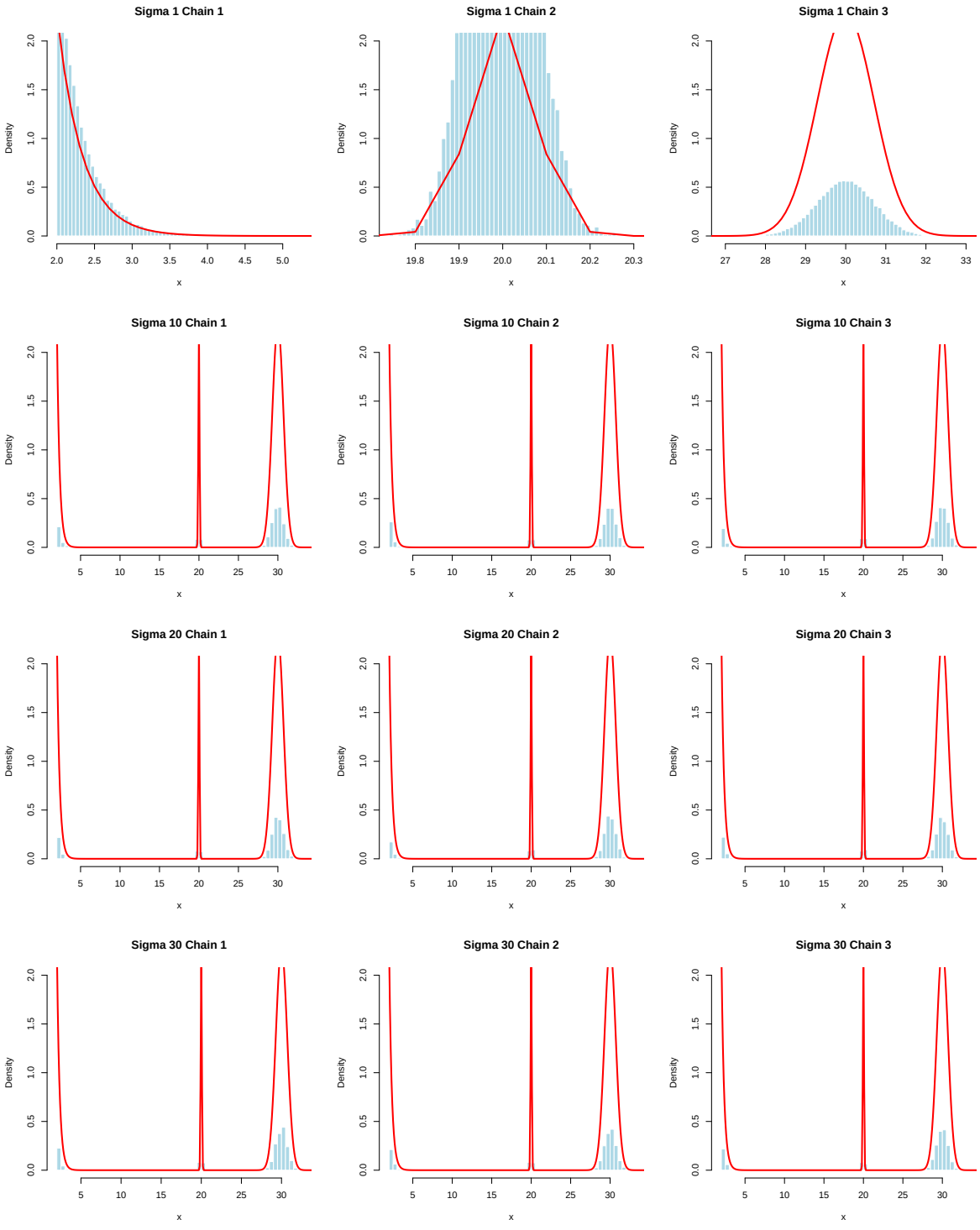


Figure 2: Histograms for four each chains

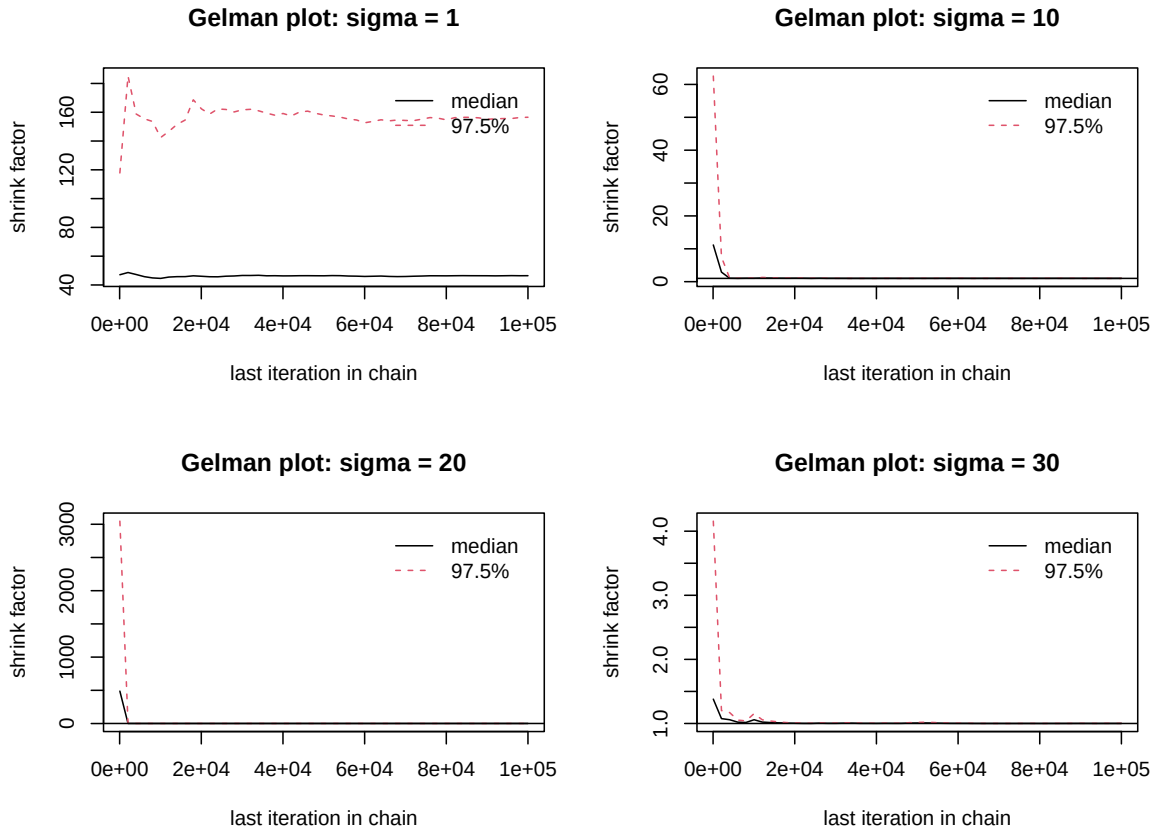


Figure 3: Gelman plots for four different variance

```
Potential scale reduction factors:

     Point est. Upper C.I.
[1,]       46.4        157

Potential scale reduction factors:

     Point est. Upper C.I.
[1,]          1       1.01

Potential scale reduction factors:

     Point est. Upper C.I.
[1,]          1       1.01
```

```
Potential scale reduction factors:

     Point est. Upper C.I.
[1,]          1          1
```

Some comments:

Visually, when $\sigma = 1$, the Markov Chains (MCs) tend to be trapped in the neighborhood of their starting point. However, in the other two cases (larger σ), the MCs are able to jump out of different modes. Despite the visually poor mixing at $\sigma = 1$, its `ESS` is high, suggesting a low autocovariance near the modes. However, as σ increases, the `ESS` first decreases, which implies the autocovariance initially rises, leading to lower mixing quality. But when $\sigma = 20$, the `ESS` rises again, indicating a reduction in autocovariance and improved mixing quality. The case for $\sigma = 30$ is similar to that of $\sigma = 20$.

In terms of convergence, for $\sigma = 1$, it is clear from the histograms that the MCs are trapped in their modes visually, and the Gelman and Rubin's convergence diagnostic is extremely high, implying no convergence. As for the other three cases, the upper CI of the Gelman and Rubin's convergence diagnostics for $\sigma = 10, 20, 30$ are closed lower than 1.1. And trace plots and histograms also provide some good evidence that they are convergent.

All in all, `ESS` is not reliable in the context of this question. And we should ask for different metrics of MCs to justify whether the chains converge and mixing quality.

(b)

I will use truncated normal distribution as proposal. Consider

$$1 - \Phi(\frac{2-\mu}{\sigma}) = \int_2^\infty \mathcal{N}(x; \mu, \sigma^2) dx,$$

it possible for us to define the truncated density:

$$q(x|y) = \frac{\mathcal{N}(x; y, \sigma^2)}{1 - \Phi(\frac{2-y}{\sigma})} \mathbb{1}_{\{x \geq 2\}},$$

which ensures the proposal must be larger than 2 almost surely, so that it must be in the support of f. Consider the acceptance rate, we have:

$$\alpha(x, y) = \frac{f(y)q(x|y)}{f(x)q(y|x)} = \frac{f(y)(1 - \Phi(\frac{2-x}{\sigma}))}{f(x)(1 - \Phi(\frac{2-y}{\sigma}))} \mathbb{1}_{\{x \geq 2, y \geq 2\}} \Rightarrow \alpha(x, y) = \min\left(1, \frac{f(y)(1 - \Phi(\frac{2-x}{\sigma}))}{f(x)(1 - \Phi(\frac{2-y}{\sigma}))} \mathbb{1}_{\{x \geq 2, y \geq 2\}}\right).$$

Due to the symmetry, we could verify that based on this proposal distribution, the MC satisfies the detailed balance condition:

$$q(y|x)\alpha(x,y) = \frac{\mathcal{N}(y;x,\sigma^2)}{1-\Phi(\frac{2-x}{\sigma})}\mathbb{1}_{\{y\geq 2\}}\min\left(1, \frac{f(y)(1-\Phi(\frac{2-x}{\sigma}))}{f(x)(1-\Phi(\frac{2-y}{\sigma}))}\mathbb{1}_{\{x\geq 2, y\geq 2\}}\right)$$
$$= \frac{\mathcal{N}(x;y,\sigma^2)}{1-\Phi(\frac{2-y}{\sigma})}\mathbb{1}_{\{x\geq 2\}}\min\left(1, \frac{f(x)(1-\Phi(\frac{2-y}{\sigma}))}{f(y)(1-\Phi(\frac{2-x}{\sigma}))}\mathbb{1}_{\{x\geq 2, y\geq 2\}}\right) = q(x|y)\alpha(y,x),$$

so that the MC will converge to the invariant distribution.

Followed by 1(a), I will choose $\sigma = 20$ in my truncated random walk. As we see the larger variance, the higher rejection rate. And the `ESS` metric is similar between $\sigma = 20$ and $\sigma = 30$.

The algorithm is similar to what we have done in random walk:

- Suppose we are at stage $t-1$, we draw a random variable u from `unif(0,1)`;
- We generate new proposal Y from the truncated random walk $q(y|x_t)$;
- If $u \leq \alpha(X_{t-1}, Y)$, we accept the proposal, namely $X_t = Y$; otherwise $X_t = X_{t-1}$.

```
Potential scale reduction factors:

     Point est. Upper C.I.
[1,]          1       1.01
```

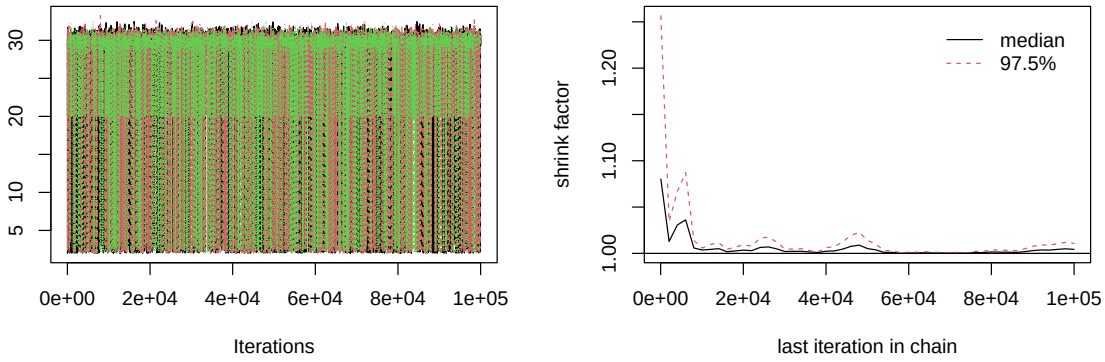


Figure 4: Improved MH-algotirhm by restricting the proposal in the support

```
Chain 1 , ESS is: 1977.857
Chain 2 , ESS is: 2087.834
Chain 3 , ESS is: 1910.452
```

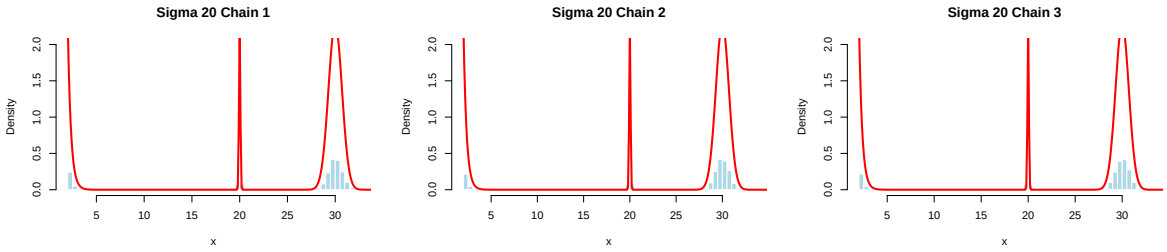


Figure 5: Histograms for Improved MH-algotirhm

Conclusion: From the traceplot, we see the chains attain almost everywhere between the lower limit and upper limit, and mix well. From the histograms, they are similar to the proposed density functions. Also from Gelman and Rubin's convergence diagnostic, the upper CI is lower than 1.1, which provides good evidence that the chains converge. Hence, we conclude that the chains converge with different start.

(c)

Pseudo Codes

- Settings: α_0, σ, $T_1, T_2, T_3, T_4, ..., T_m$.
- Step 1: Draw u from `unif (0,1)`.
- Step 2: If $u \leq \alpha_0$, then we do MCMC in each chain and update X_i^{n+1} from X_i^n (denote the subscript is the location of chain, and the superscript is the stage of the chain). I choose the random walk ($\mathcal{N}(y; x, \sigma_i^2)$) as the proposal distribution, so the acceptance rate is
$$\alpha_1(x, y) = \min\{1, \frac{f^{1/T_i}(y)}{f^{1/T_i}(x)}\},$$
for $i \in \{1, 2, 3, ..m\}$
- Else, we swap the chain: by randomly choosing one neighbor chain (e.g. $(X_i^n, X_{i+1}^n) \to (X_{i+1}^{n+1}, X_i^n)$), this proposal is symmetric, so we accept this proposal with probability

$$\alpha_2 = \min\{1, \frac{f^{1/T_{i+1}}(X_i^n) f^{1/T_i}(X_{i+1}^n)}{f^{1/T_{i+1}}(X_{i+1}^n) f^{1/T_i}(X_i^n)}\}.$$

In terms of the transition kernel, we have two transition processes. Suppose we have m parallel chains

Within the chain:

for i in 1: m do

- We draw u_1 from `unif(0,1)`.
- Draw y_i from $q(y_i|x_i^{n-1})$ for each chain
- If $u_1 \leq \alpha_1(x_i^{n-1}, y_i)$, then we accept the proposal within the chain . Otherwise, we reject the proposal and set $X_i^n = X_i^{n-1}$.

end do

Swamping the chain

- Suppose we are at state n.
- We draw one random variable u_2 from `unif(0,1)`.
- Uniformly choose the one neighborhood pair chain to swap, namely $j \in \{1, 2, 3....., m-1\}$.
- Suppose $u_2 \leq \alpha_2$, then we exchange the state between chains, otherwise, we will not swap the chain and keep state the same as before.

In my experiment, I used $\alpha_0 = 0.5$, $T_s = (1, 20, 100, 200, 500)^T$, $\sigma = (0.1, 0.5, 1, 2, 5)$, and ran 5 chains.

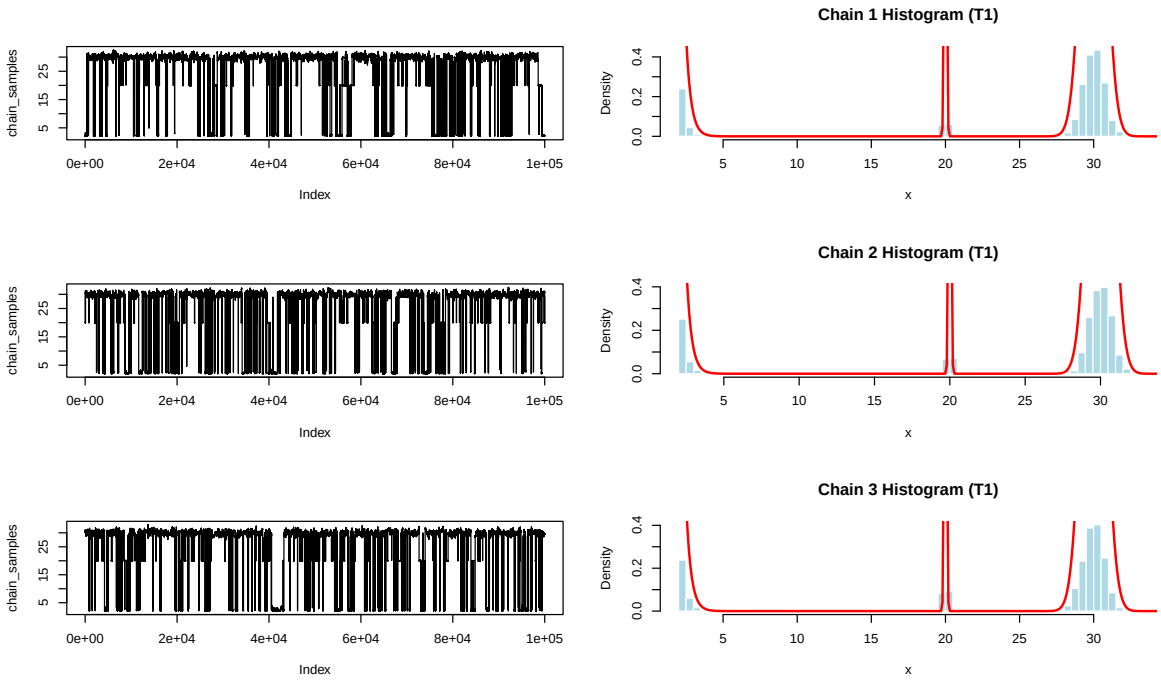


Figure 6: Compare with the true density

We then do the convergence diagnostic for this chain,

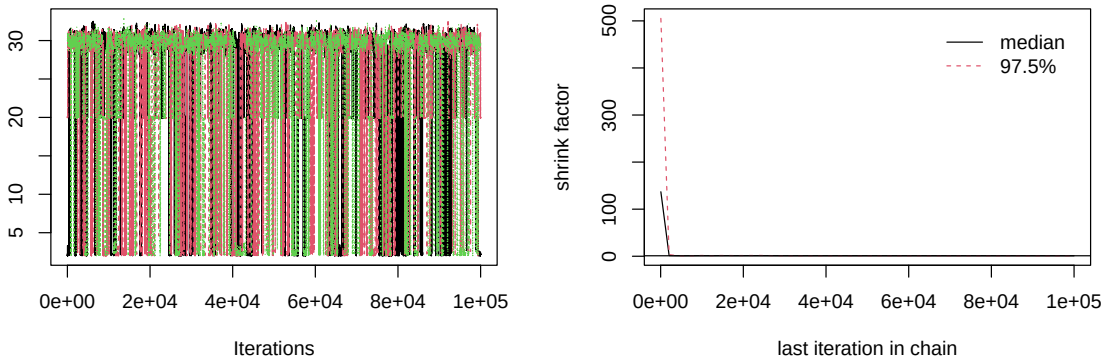
```
The ESS is 270.435 where start point is 2

The ESS is 291.5144 where start point is 20

The ESS is 288.4866 where start point is 30

Potential scale reduction factors:

     Point est. Upper C.I.
[1,]       1.01       1.02
```



From the diagonize of the cold chain, namely, $f(x)^{1/1} = f(x)$, the mixing quality is not well as ESS is low but metric Gelman and Rubin's convergence diagnostic is smaller than 1.1, suggesting a strong evidence the chain is convergent. Look back to the histogram against density plot, the frequency plot fits well to the density, which also provides strong evidence that the chain is convergent and is not sensitive to the start point.

(d)

Convergence

In the initial phase of the Random Walk in 1(a), using small σ values for the proposal step failed to achieve convergence, primarily because the multimodal nature of the target density function led the Markov chains to become trapped in local modes. By increasing the step size, particularly when σ was set to $10, 20$, and 30, the resulting histograms showed significantly

improved sampling efficacy, indicating that these σ values were optimal to some extent. Subsequently, in 1(b), the sampling process was further refined by restricting the proposed values (y) to the support of the target density, f. However, the most critical enhancement was the implementation of Parallel Tempering 1(c); by intentionally starting the chains at the modes of f, the method enabled the Markov chains to effectively explore the entire support of the distribution. Ultimately, the frequency plot showed an excellent fit to the density function f, and concurrently, the Gelman and Rubin's convergence diagnostic was found to be less than 1, providing strong evidence of very good chain mixing and convergence, successfully overcoming the challenges posed by multimodality.

Mixing quality

In 1(a), when $\sigma = 1$, the `ESS` is very high, which seems to provide evidence on good mixing. But from trace plot, three chains are trapped in the neighborhood of modes, so the `ESS` here is misleading. The `ESS` for $\sigma = 10$ and parallel tempering are similar and are around 200-300, which suggests these chains are not mixing well. The `ESS` for the remaining cases are around 1000-2000, which suggest relative good mixing compared to the other cases.

Computational cost

Compare to random walk proposal in 1(a), the truncated proposal I used in 1(b) is much more efficient as it is possible for random walk to generate proposals that is out of the support of the f and these samples cannot be accepted, but the truncated proposal is not. The method in 1(b) also attains higher `ESS` compared to 1(a) in the same number of iterations. The parallel tempering requires us to run 5 chains at the same time, which is computationally expensive compared to the previous methods.

(e)

We can easily figure out $\mathbb{E}(X)$ via `R`, which is 25.18417 and is chosen as our baseline (red line) to compare the estimator of each chain.

We directly use samples generated by Markov chain to figure out $\mathbb{E}(X)$, we give the formula of our confidence interval:

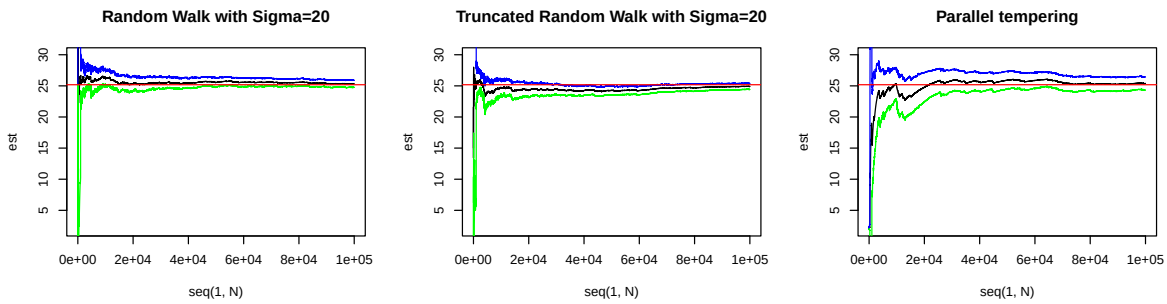
$$[\mu - z_{0.975} \cdot \frac{\tilde{\sigma}}{n}, \mu + z_{0.975} \cdot \frac{\tilde{\sigma}}{n}] = \hat{I}_n^n \pm 1.96 MCSE(\hat{I}_n),$$

where $\tilde{\sigma}$ is the asymptotic variance given by `mcmcse` package in `R` . As for implementing, we choose at first $n = 1000$ steps as burn-in stage. During this stage, we calculate the typical mean and standard deviation of our samples. The rest time we calculate the MC estimator and MC standard error using `mcmcse`.

```
The estimator by this chain is: 25.3402 and ci is: [ 24.78284 , 25.89757 ]
```

```
The estimator by this chain is: 24.89868 and ci is: [ 24.40459 , 25.39277 ]

The estimator by this chain is: 25.38916 and ci is: [ 24.35523 , 26.4231 ]
```



Conclusion

After $n = 1e5$ iterations, all of the three methods converge closely to the true value of $\mathbb{E}(X)$, which is marked as red line. We observe that the convergence to $\mathbb{E}(X)$ is positively correlated with the convergence of the Markov Chains (MCs), which is measured by the `ESS`. Compared to case 1(a), case 1(b) yields a smaller interval and is thus more precise. Furthermore, the Confidence Interval (CI) generated by 1(c) is notably wider than the other two methods, due to the very high variance of the generated samples and low `ESS`. In our setting, Parallel Tempering converges slower than the Random Walk and Truncated Random Walk methods with $\sigma = 20$, mainly because its computational expense is much higher compared to the previously mentioned methods.

Question 2

(a)

(i)

Let me explain how the optimization is performed. We notice that our loss function is not differentiable at q. So to avoid some incidence, I choose to use "Nelder-Mead" rather than quicker method "BFGS". The trival vector $(0, 0, 0)$ is chosen as my start point. And the optimization converge ultimately.

```
When tau is 0.1 ,the empirical loss is 0.6964838 and the estimator is -
1.229301 -0.7937421 4.543845 convergence: 0
When tau is 0.5 ,the empirical loss is 1.904715 and the estimator is -11.42951 -
0.5723921 6.43675 convergence: 0
```

```
When tau is 0.9 ,the empirical loss is 1.133979 and the estimator is -27.63414 -
0.200334 9.303498 convergence: 0
```

(ii)

The output of empirical loss of test data set is stated as follow:

```
When tau is 0.1 ,the empirical loss is: 0.7835174
When tau is 0.5 ,the empirical loss is: 1.977712
When tau is 0.9 ,the empirical loss is: 1.245324
```

(iii)

Let's have a recap, the choice of τ is $(0.1, 0.5, 0.9)$. And the displayed results in my experiments are very close to the initial setting, implying some effectiveness of our method.

```
[1] 0.09210526 0.48026316 0.88815789
```

(b)

(i)

Let me explain how the optimization is performed. We notice that our loss function is not differentiable at q. So to avoid some incidence, I choose to use "Nelder-Mead" rather than quicker method "BFGS". The trivel vector $(0, 0, 0, 0, 0, 0)$ is chosen as my start point as we have six parameters. And the optimization doesn't converge ultimately.

```
When tau is 0.1 ,the empirical loss is 0.6583881 and the estimator is -
1.017769 0.3667278 -0.0528613 0.004520817 0.706361 -0.1956024 convergence: 1
When tau is 0.5 ,the empirical loss is 1.880985 and the estimator is -0.2010836 -
0.9668291 -0.1125912 0.01950876 0.7236458 0.03394346 convergence: 1
When tau is 0.9 ,the empirical loss is 0.8436352 and the estimator is -
0.472779 1.760224 0.08120124 0.00781569 0.8769808 -0.421138 convergence: 1
```

(ii)

```
When tau is 0.1 ,the empirical loss is: 1.970295
When tau is 0.5 ,the empirical loss is: 17.65133
When tau is 0.9 ,the empirical loss is: 7.628274

[1] 0.09210526 0.46052632 0.87500000
```

(c)

Given that $\lambda \in [10^{-4}, 10^{-1}]$, we choose pick 100 samples, where the neighbor distance is 0.001.

The optimization setting is the same as that in 2(b). And we spot that very few training procedures converge when $\tau = 0.1, 0.5$. But it improves when $\tau = 0.9$.

```
About 2  percent of experiments converge in the training
When tau is 0.1 , the best lambda is 0.0111
About 3.8  percent of experiments converge in the training
When tau is 0.5 , the best lambda is 0.0051
About 24.2  percent of experiments converge in the training
When tau is 0.9 , the best lambda is 0.0081

The empirical loss is 0.6643584 when tau is 0.1 Convergence: 1
The coefficients are 1.905407 1.779397 -0.002839653 -0.006681427 0.6250297 -
0.3886277
The empirical loss is 1.598406 when tau is 0.5 Convergence: 1
The coefficients are -0.1990285 2.544147 1.054229 0.002767224 0.6893941 -
0.5768584
The empirical loss is 1.046116 when tau is 0.9 Convergence: 1
The coefficients are 1.401273 3.032821 2.258348 0.006443642 0.617555 -0.6807364

When tau is 0.1 ,the empirical loss is: 0.7580092
When tau is 0.5 ,the empirical loss is: 1.847137
When tau is 0.9 ,the empirical loss is: 0.9663356

[1] 0.1118421 0.5723684 0.9210526
```

(d)

Coefficient magnitudes:

The Model (b) produces very large and unstable coefficients, not because the data exhibit strong nonlinear structure, but because the non-smooth quantile loss causes the Nelder–Mead optimiser to frequently fail in higher dimensions, leading to optimisation-driven coefficient inflation. In contrast, ridge regularisation stabilises the optimisation and shrinks coefficients to reasonable scales, while the baseline linear model naturally remains stable due to its low dimensionality.

Test loss:

- Compare Model (a) and Model (b)

As Model(b) introduces more variables than Model (a), it is possible that Model (b) contains too much information. And the testing losses show that in all value of τ, the testing losses of Model (b) are much higher than those of Model(b), especially then $\tau = 0.5, 0.9$. From the convergence in Model (b), the more parameters make it harder to converge an optimal parameters, trapping in the local optimization and leading to high testing loss.

- Compare Model (b) and Model (c)

Model (b) and Model (c) shares the same number of parameter, but Model (c) introduces ridge regression. In all three cases of τ, the testing losses of Model (c) is much smaller than those of Model (b).

Overall, the Model (b) is the worst model among the three. Model (a) fits well, and Model (c) helps to improve Model (c) in the same dimensions.

Overfitting:

- Model (a) The training loss and the testing loss is very close in both three cases, suggesting a good fitting.
- Model (b) Although the testing loss is about 2 for $\tau = 0.1$, it is extremely large when $\tau = 0.5, 0.9$, but the training loss is low, suggesting overfitting in this model.
- Model (c) Although the loss is not convergent in training, compared to the Model (b), the difference between the training and testing loss is low, suggesting no overfitting in this model. Also, it is a large improvement compared to the Model(b).

Empirical coverage:

- Model (a) The result is $(0.09210526, 0.48026316, 0.88815789)$, very closed to our baseline. As we mentioned before, this model fits well.

- Model (b) The result is $(0.09210526, 0.46052632, 0.87500000)$, although the first and the third item are very closed to the baseline, the middle item, with the largest testing loss, is far away from our baseline, performing bad in the experiment, but coincide with the test loss.
- Model (c) The result is $(0.1118421, 0.5723684, 0.9210526)$, except the middle part, the other estimated quantiles are close to our baseline. The reason perhaps is the non-convergence of the training part. This model improves the performance in some sense, but it is still unstable as one outlier appears.

Conclusion:

The main performance differences arise not only from model complexity but from the interaction between flexibility and optimization stability. The Model (b) fails mainly due to non-convergence, the linear model is stable but limited, and the ridge model provides the best overall balance of bias, variance and reliable optimization.

(e)

(i)

Not necessarily, it is not guaranteed. Firstly, in the optimization procedure, the models in (b) and (c) don't converge to optimal solutions, so that it may cause large bias in estimating the true parameter and lead to the crossing of other quantile models. Secondly, there is no restriction of forcing one model to be larger than the other, as we didn't embed any another properties to this family of models. Optimize independently may be an another cause.

(ii)

Rather than optimize independently, we attempt to optimize this family of models together. We also need to add some penalty if our models against the quantile rule, so we may do similar what we do in ridge regression model.

$$R_{\lambda}^{Ridge}(\beta) = \frac{1}{n_{train}} \sum_{\tau} \sum_{i \in train} l_{\tau}(Y_i, f(X_i; \beta_{\tau})) + \lambda \mathcal{P}(\beta),$$

where $\mathcal{P}(\beta)$ to punish those samples against monotonicity.

Suppose we want to embed our model with some property (like translation, scale, etc. which are equation), we simply put them into loss function. However, we need to avoid such equation, for $\tau_1 = 0.1, \tau_2 = 0.5$, we need to make

$$f(X_i; \beta_{\tau_1}) - f(X_i; \beta_{\tau_2}) < 0$$

difference as large as possible without violating the basic purpose to estimate the quantile. On the other hand, we could investigate positive part $X_+ = max(X, 0)$ as small as possible:

$$\mathcal{P}(\beta) = \lambda_1 \sum_{i \in train} \{[f(X_i; \beta_{\tau_1}) - f(X_i; \beta_{\tau_2})]_+\}^2 + \lambda_2 \sum_{i \in train} \{[f(X_i; \beta_{\tau_2}) - f(X_i; \beta_{\tau_3})]_+\}^2 + \lambda_3 \sum_{i \in train} \{[f(X_i; \beta_{\tau_1}) - f(X_i; \beta_{\tau_3})]_+\}^2,$$

where $\lambda_{1,2,3}$ are positive number and $\lambda_1 + \lambda_2 + \lambda_3 = 1$. To make the loss small is to make positive part close to 0, which satisfies our requirement.

Code appendix

```
library('coda')
f <- function(x) ifelse(x >= 2, exp(-3*(x - 2)) + exp(-(x - 30)**2) + exp(-(x - 20)**2/0.01)
#Metropolis-Hasting Algorithm for random walk
rq <- function(x, sigma) rnorm(1, mean = x, sd=sigma)
mh <- function(x0, fun, rq, sg, n){
  xall <- rep(NA,n+1)
  x <- x0
  xall[1] <- x


  for (t in seq(2,n+1)){
    x <- x
    y <- rq(x,sg)

    if (log(runif(1))<=log(fun(y))-log(fun(x))){
      x <- y
      }
    xall[t] <- x
    }

  return(xall)
}

set.seed(49)
num <- 3
na <- 1e5
```

```r
x0 <- c(2, 20, 30)
sigma1 <- 1
sigma2 <- 10
sigma3 <- 20
sigma4 <- 30
chains1_a <- lapply(1:num, function(i) mcmc(mh(x0[i], f, rq, sigma1, na)))
chains2_a <- lapply(1:num, function(i) mcmc(mh(x0[i], f, rq, sigma2, na)))
chains3_a <- lapply(1:num, function(i) mcmc(mh(x0[i], f, rq, sigma3, na)))
chains4_a <- lapply(1:num, function(i) mcmc(mh(x0[i], f, rq, sigma4, na)))
sigmas <- c(sigma1, sigma2, sigma3, sigma4)
chain_a <- c(chains1_a, chains2_a, chains3_a, chains4_a)
for (i in 1:length(chain_a)){

  current_chain_a <- chain_a[[i]]

  ESS_a <- effectiveSize(current_chain_a)

  sigma_index <- floor((i - 1) / 3) + 1

  sigma_val <- sigmas[sigma_index]

  cat("Chain", i, ": Sigma is", sigma_val, ", ESS is:", ESS_a, "\n")
}
par(mfrow = c(2, 2))
traceplot(chains1_a, main = paste("Traceplot: sigma =", 1))
traceplot(chains2_a, main = paste("Traceplot: sigma =", 10))
traceplot(chains3_a, main = paste("Traceplot: sigma =", 20))
traceplot(chains4_a, main = paste("Traceplot: sigma =", 30))
hist_mc <- function(chains, sg){
for (i in 1:length(chains)) {
    chain_samples <- chains[[i]]

    hist(chain_samples,
         breaks = 50,
         probability = TRUE,
         main = paste("Sigma", sg, "Chain", i),
         xlab = "x",
         ylab = "Density",
         ylim = c(0, 2),
         col = "lightblue",
         border = "white")
```

```
    lines(x = xnodes,
          y = k * f(xnodes),
          col = "red",
          lwd = 2)
}
}
par(mfrow = c(4, 3))
xnodes <- seq(2, 50, 0.1)
k <- integrate(f, lower = 2, upper = Inf)$value
hist_mc(chains1_a,sigma1)
hist_mc(chains2_a,sigma2)
hist_mc(chains3_a,sigma3)
hist_mc(chains4_a,sigma4)
par(mfrow = c(2, 2))
chains1_a <- mcmc.list(chains1_a)
chains2_a <- mcmc.list(chains2_a)
chains3_a <- mcmc.list(chains3_a)
chains4_a <- mcmc.list(chains4_a)

gelman.plot(chains1_a, main = paste("Gelman plot: sigma =", 1), auto.layout =  FALSE)
gelman.plot(chains2_a, main = paste("Gelman plot: sigma =", 10), auto.layout =  FALSE)
gelman.plot(chains3_a, main = paste("Gelman plot: sigma =", 20), auto.layout =  FALSE)
gelman.plot(chains4_a, main = paste("Gelman plot: sigma =", 30), auto.layout =  FALSE)
gelman.diag(chains1_a)
gelman.diag(chains2_a)
gelman.diag(chains3_a)
gelman.diag(chains4_a)
library('truncdist')
# Improved MH for support
set.seed(355)
MH <- function(f,init, n){
  xall <- rep(NA,n)
  x <- init
  xall[1] <- x
  for (t in seq(2,n)){
    y <- rtrunc(n=1, 'norm', a=2, mean = x, sd = 20)
    if (runif(1)<=f(y)*pnorm(2, mean=x, sd = 20, lower.tail = FALSE)/(f(x)*pnorm(2, mean = y
      x<-y
    }
    xall[t] <- x
    }
  return(xall)
```

```
}
par(mfrow = c(1, 2))
nb <- 1e5
x1 = c(2, 20, 30)
chains1_b <- lapply(1:3, function(i) mcmc(MH(f, x1[i], nb)))
traceplot(chains1_b)
gelman.diag(chains1_b)
gelman.plot(chains1_b, auto.layout =  FALSE)
abline(h = 2, col = 'gray')
for (i in 1:length(chains1_b)){

  current_chain <- chains1_b[[i]]

  ESS <- effectiveSize(current_chain)

  cat("Chain", i, ", ESS is:", ESS, "\n")
}
par(mfrow=c(1,3))
hist_mc(chains1_b, sg=20)
paralleltemp <- function(num.steps, Ts, sg, init, threshold) {
  M <- length(Ts)
  samples <- matrix(NA, nrow = num.steps, ncol=M)

  X <- rep(init, M)

  for (n in 1:num.steps) {
    if (runif(1) <= threshold){
      # we are going to do a x-step
      for (m in 1:M) {
        y <- X[m] + sg[m]*rnorm(1)
        alpha <- f(y)^(1/Ts[m]) / f(X[m])^(1/Ts[m])

        if (runif(1) <= alpha) {
          X[m] <- y
        }
      }
    }
       else{
        ### we are going for a swap

        m <- sample(1:(M-1), 1)
```

```
        num = (f(X[m])^(1/Ts[m+1])*f(X[m+1])^(1/Ts[m]))
        denom = (f(X[m])^(1/Ts[m])*f(X[m+1])^(1/Ts[m+1]))

        if (runif(1) <= num/denom) {
          X[c(m, m+1)] = X[c(m+1, m)]
        }
     }
        samples[n, ] = X
  }

  return(samples)
}
set.seed(80)
Ts <- c(1, 20, 100, 200, 500)
sigmas <- c(0.1, 0.5, 1, 2, 5)
nstep <- 1e5
init <- c(2, 20, 30)
chains5 <- lapply(1:length(init), function(i) paralleltemp(nstep, Ts, sigmas, init[i], 0.5))
xnodes <- seq(2, 50, 0.1)
k <- integrate(f, lower = 2, upper = Inf)$value
#plot(chains5[1][,1], type='l')
par(mfrow = c(3, 2))
for (i in 1:length(chains5)) {
    chain_samples <- chains5[[i]][, 1]

    plot(chain_samples, type='l')

    hist(chain_samples,
         breaks = 50,
         probability = TRUE,
         main = paste("Chain", i, "Histogram (T1)"),
         xlab = "x",
         ylab = "Density",
         col = "lightblue",
         border = "white")

    lines(x = xnodes,
          y = k * f(xnodes),
          col = "red",
          lwd = 2)
}
par(mfrow = c(1, 2))
```

```r
chain_T1_run1 <- mcmc(chains5[[1]][, 1])
chain_T1_run2 <- mcmc(chains5[[2]][, 1])
chain_T1_run3 <- mcmc(chains5[[3]][, 1])
T1_mcmc_list <- mcmc.list(chain_T1_run1, chain_T1_run2, chain_T1_run3)
cat("The ESS is", effectiveSize(chain_T1_run1), "where start point is", 2, "\n")
cat("The ESS is", effectiveSize(chain_T1_run2), "where start point is", 20, "\n")
cat("The ESS is", effectiveSize(chain_T1_run3), "where start point is", 30, "\n")
gelman.diag(T1_mcmc_list)
par(mfrow = c(1, 2))
traceplot(T1_mcmc_list)
gelman.plot(T1_mcmc_list, auto.layout = FALSE)
library("mcmcse")
xf <- function(x) x * f(x) / k
true <- integrate(xf, lower = 2, upper = Inf)
mc_ci <- function(chain){
  n <- length(chain)
  est <- rep(0, n)
  se <- rep(0, n)
  chain_vector <- as.vector(chain)
  for(i in 1:n){
    if (i >= 1000){
      mcse_result <- mcse(x = chain_vector[1:i], method = "bm")
      I_hat <- mcse_result$est
      MCSE_est <- mcse_result$se

      est[i] <- I_hat
      se[i] <- MCSE_est
    }
  else{
      I_hat <- mean(chain_vector[1:i])
      MCSE_est <- sd(chain_vector[1:i])

      est[i] <- I_hat
      se[i] <- MCSE_est

  }
  }
    #cat(length(est), length(se), "\n")
    cat("The estimator by this chain is:", I_hat, "and ci is:", "[", I_hat-1.96*MCSE_est,","

    para <- data.frame(I = est, se = se)
    return(para)
```

```
}
ci_plot <- function(chain, title){
  par <- mc_ci(chain)
  N <- length(chain)
  est <- par$I
  sd  <- par$se

  #Plot the running mean and confidence interval
  plot(seq(1, N), est, type='l', ylim = c(2, 30), main = title)
  lines(seq(1, N), est + 1.96 * sd , col='blue')
  lines(seq(1, N), est - 1.96 * sd, col='green')
  abline(h = true$value, col = 'red')
}
par(mfrow = c(1, 3))
set.seed(42)
n <- 1e5
#From 1(a)
chain1_d <- mh(15, f, rq, 20, n)
#From 1(b)
chain2_d <- MH(f, 15, n)
#From 1(c)
chain3_d <- paralleltemp(n, Ts, sigmas, 2, 0.5)[,1]

#cat("The estimator by chain 1 is:", mean(chain1_d), "and ci is:", "[", mean(chain1_d)-1.96*s
#cat("The estimator by chain 2 is:", mean(chain2_d), "and ci is:", "[",mean(chain2_d)-1.96*sd
#cat("The estimator by chain 3 is:", mean(chain3_d), "and ci is:", "[",mean(chain3_d)-1.96*sd

par(mfrow = c(1, 3))
ci_plot(chain1_d, "Random Walk with Sigma=20")
ci_plot(chain2_d, "Truncated Random Walk with Sigma=20")
ci_plot(chain3_d, "Parallel tempering")
library('MASS')
set.seed(46)
data <- data.frame(y = Boston$medv,
                   x1 = Boston$lstat,
                   x2 = Boston$rm)

n <- length(data$y)

tr_index <- sample(1:n, floor(n * 0.7))
```

```r
data_tr <- data[tr_index, ]

data_test <- data[-tr_index,]

loss <- function(tau, y, q){
  ifelse(y > q, tau * (y - q), (tau - 1) * (y - q))
}

empirical_loss <- function(beta, tau, x1, x2, y){
  beta1 <- beta[1]
  beta2 <- beta[2]
  beta3 <- beta[3]


  q <- beta1 + beta2 * x1 + beta3 * x2

  return(mean(loss(tau, y, q)))

  }
init_value <- c(0, 0, 0)
tau <- c(0.1, 0.5, 0.9)
beta_es <- matrix(data = 0, nrow = 3, ncol = 3)

for (i in 1 :length(tau)){
  opt_result <- optim(par = init_value, fn = empirical_loss, tau = tau[i], x1 = data_tr$x1,
  x2 = data_tr$x2, y = data_tr$y, method="Nelder-Mead")
  beta_es[i, ] <- opt_result$par
  cat("When tau is", tau[i], ",the empirical loss is", opt_result$value, "and the estimator i

}
matrix_test <- matrix(data = 1, nrow = length(data_test$y), 3)
matrix_test[,2] <- data_test$x1
matrix_test[,3] <- data_test$x2

matrix_y <- matrix(data = 1, nrow = length(data_test$y), ncol = length(tau))
for (i in 1:3){
  matrix_y[,i] <- data_test$y
}
fx <- matrix_test %*% t(beta_es)

for (i in 1:length(tau)){
  los <- empirical_loss(beta_es[i,], tau[i], data_test$x1, data_test$x2, data_test$y)
```

```
  cat("When tau is", tau[i], ",the empirical loss is:", los, "\n")
}
compare <- matrix_y <= fx
count_y_smaller_fx <- colSums(compare)/length(data_test$y)

count_y_smaller_fx

empirical_risk_b <- function(beta, tau, x1, x2, y){
  beta1 <- beta[1]
  beta2 <- beta[2]
  beta3 <- beta[3]
  beta4 <- beta[4]
  beta5 <- beta[5]
  beta6 <- beta[6]


  q <- beta1 + beta2 * x1 + beta3 * x2 + beta4 * x1 ** 2 + beta5 * x2 ** 2 + beta6 * x1 * x2

  return(mean(loss(tau, y, q)))
   }
init_value_b <- c(0, 0, 0, 0, 0, 0)
tau <- c(0.1, 0.5, 0.9)
beta_es_b <- matrix(data = 0, nrow = 3, ncol = 6)
#beta_es_b

for (i in 1 :length(tau)){
  opt_result_b <- optim(par = init_value_b, fn = empirical_risk_b, tau = tau[i], x1 = data_tr
  x2 = data_tr$x2, y = data_tr$y, method="Nelder-Mead")
  beta_es_b[i , ] <- opt_result_b$par
  cat("When tau is", tau[i], ",the empirical loss is", opt_result_b$value, "and the estimator
  opt_result_b$convergence
}

matrix_test_b <- matrix(data = 1, nrow = length(data_test$y), ncol = 6)
matrix_test_b[,2] <- data_test$x1
matrix_test_b[,3] <- data_test$x2
matrix_test_b[,4] <- matrix_test[,2] **2
matrix_test_b[,5] <- matrix_test[,3] **2
matrix_test_b[,6] <- matrix_test[,2] * matrix_test[,3]

fx_b <- matrix_test_b %*% t(beta_es_b)
```

```r
for (i in 1:length(tau)){
  los_b <- empirical_loss(beta_es_b[i,], tau[i], data_test$x1, data_test$x2, data_test$y)
  cat("When tau is", tau[i], ",the empirical loss is:", los_b, "\n")
}

compare_b <- matrix_y <= fx_b
count_y_smaller_fx_b <- colSums(compare_b)/length(data_test$y)
count_y_smaller_fx_b


set.seed(46)
lambda_list <- seq(1e-4, 1e-1, 0.001)
k_fold <- 5
folds <- sample(rep(1:k_fold, length.out=length(data_tr$y)))
empirical_risk_c <- function(par, lambda, tau, x1, x2, y){
  beta1 <- par[1]
  beta2 <- par[2]
  beta3 <- par[3]
  beta4 <- par[4]
  beta5 <- par[5]
  beta6 <- par[6]


  q <- beta1 + beta2 * x1 + beta3 * x2 + beta4 * x1 ** 2 + beta5 * x2 ** 2 + beta6 * x1 * x2

  return(mean(loss(tau, y, q)) + lambda * sum(par**2))
}
best_lambda <- function(tau){
  init_value_c <- rep(0, 6)
  cv_mse <- numeric(length(lambda_list))
  convergence <- c()

  for (j in 1: length(lambda_list)){
      errs <-numeric(k_fold)

      for (i in 1:k_fold){
        tr <- folds != i
        ts <- folds == i

        tr_da <- data.frame(y = data_tr$y[tr], x1 = data_tr$x1[tr], x2 = data_tr$x2[tr])
        ts_da <- data.frame(y = data_tr$y[ts], x1 = data_tr$x1[ts], x2 = data_tr$x2[ts])
```

```
        opt_result_c <- optim(par = init_value_c, fn = empirical_risk_c, lambda = lambda_list

        par <- opt_result_c$par

        err <- empirical_risk_c(par, lambda = 0, tau, ts_da$x1, ts_da$x2, ts_da$y)

        errs[i] <- err
        #cat(par, "\n")

        #cat("Convergence:", opt_result_c$convergence, "\n")
        convergence <- c(convergence, opt_result_c$convergence)
      }
      ms <- mean(errs)
      cv_mse[j] <- ms

  }
  cat("About", 100 - 100 * mean(convergence), " percent of experiments converge in the train
  return(cv_mse)
}
lambda_test <- numeric(length(tau))
for (i in 1:length(tau)){
  min_index <- which.min(best_lambda(tau[i]))
  min_lambda <- lambda_list[min_index]
  lambda_test[i] <- min_lambda
  cat("When tau is", tau[i], ","the best lambda is", min_lambda, "\n")
}
beta_es_c <- matrix(0, nrow =6, ncol = 3)
init_value_c <- rep(1, 6)
for (j in 1: length(lambda_test)){

      opt_result_c2 <- optim(par = init_value_c, fn = empirical_risk_c, lambda = lambda_test

      beta_es_c[,j] <- opt_result_c2$par
      cat("The empirical loss is", opt_result_c2$value, "when tau is", tau[j], "Convergence:"
      cat("The coefficients are", opt_result_c2$par, "\n")

}
for (i in 1:length(tau)){
  los_b <- empirical_risk_b(beta_es_c[,i], tau[i], data_test$x1, data_test$x2, data_test$y)
  cat("When tau is", tau[i], ",the empirical loss is:", los_b, "\n")
}
```

```r
fx_c <- matrix_test_b %*% beta_es_c

compare_c <- matrix_y <= fx_c
count_y_smaller_fx_c <- colSums(compare_c)/length(data_test$y)
count_y_smaller_fx_c
```